

Объектная модель данных и язык объектных запросов OQL

Постреляционные базы данных
Лекция №5

Виноградова М.В.
МГТУ им. Н.Э. Баумана

Темы лекции

- Стандарт ODMG
- Объектные базы данных
- Основные понятия объектной модели
- Описание объектной модели на языке ODL
- Преобразование ER-модели к объектной
- Язык объектных запросов OQL
- Встраивание объектных запросов в приложение на объектно-ориентированном языке

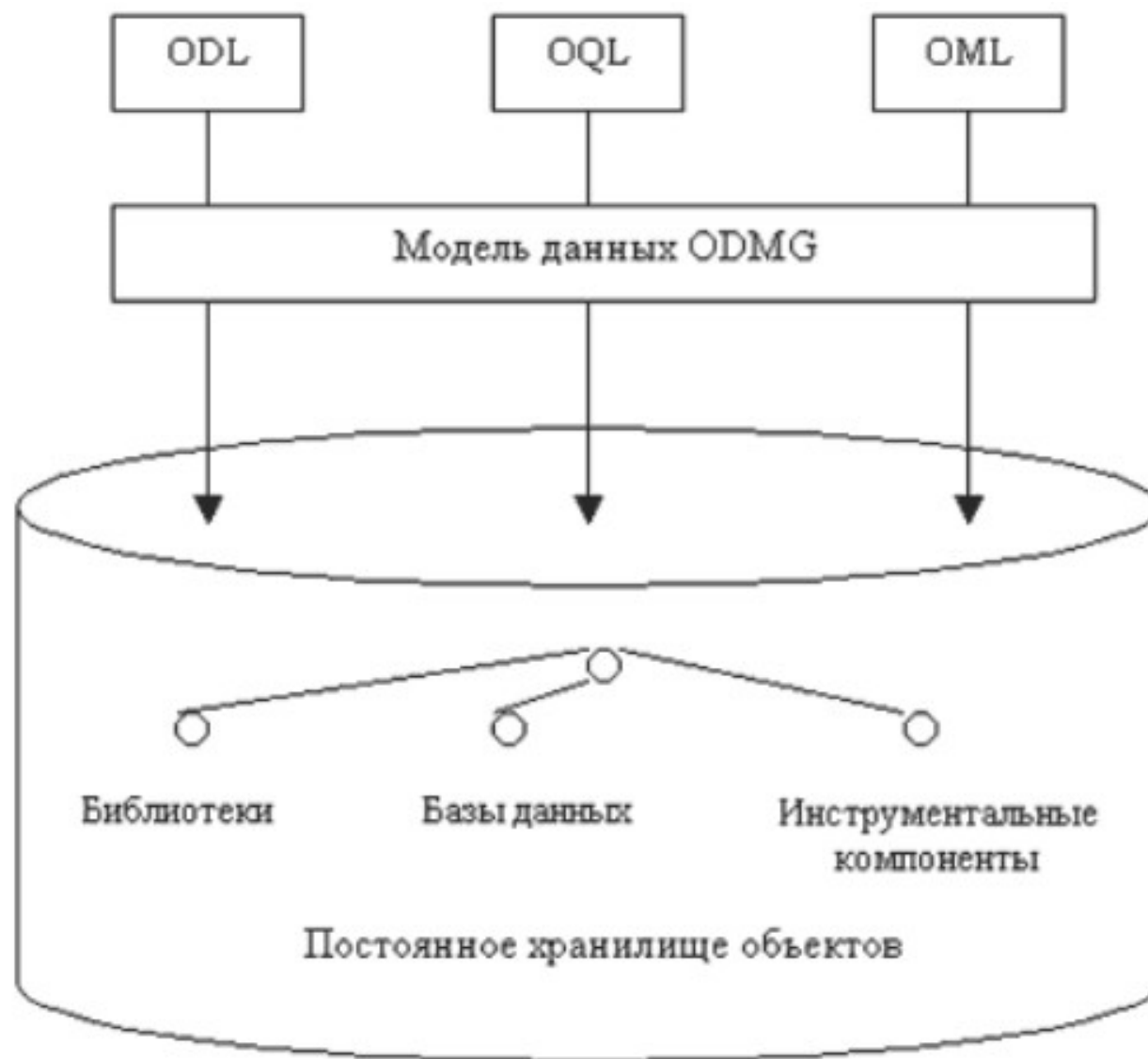
Стандарты ODMG

- 1989 - консорциумом OMG (*Object Management Group*)
- 1991 - консорциум ODMG (*Object Database Management Group*, позднее *Object Data Management Group*)
- Выботка стандартов для ООБД на основе объектной модели OMG COM (*Core Object Model*) .
- ODMG-93 и ODMG-2003 – наиболее известные стандарты.
- После 2001 стандартом занимается OMG.

ОО СУБД

- **Cache` Intersystems** – полная объектная модель (в т.ч. методы) как одно из представлений данных.
- **ConceptBase**
- **OrientDb**
- **Objectivity/DB**
- **Versant**

Архитектура ODMG



Языки стандарта ODMG

- ODL - язык определения данных
 - Виртуальный
 - Задаёт возможности
 - Используются ООЯП для СУБД
- OQL - язык объектных запросов
 - Основан на SQL
 - Встраивается в ОО ЯПВУ
- OML - язык манипулирования объектами
 - Интегрирован или расширяет ОО ЯПВУ

Объектная модель баз данных

- Соответствует парадигме объектно-ориентированных языков программирования.
- Поддерживает инкапсуляцию, наследование и полиморфизм.
- В объектной базе данных хранят объекты классов.
- Класс объекта определяет его структуру, включая атрибуты, методы и связи.

Возможности объектной СУБД

- поддерживает сложную систему типов, в том числе атомарные, структуры, коллекции и ссылки и предоставляет средства для их описания;
- разделяет работу с литералами (неизменяемыми значениями любых типов) и объектами классов;
- обеспечивает идентификацию объектов;
- позволяет определять иерархии классов и интерфейсов.

Объекты и классы

- **Объект** — экземпляр некоторого класса.
- **Класс** — специальный тип данных, называемый также объектным типом. Он задает интерфейс относящихся к нему объектов.
- Объект имеет атрибуты, методы и связи. Их перечень, названия, сигнатуры и типы указаны в описании класса.
- Атрибуты и связи называют свойствами, а свойства и операции — характеристиками объектного типа или объектов данного типа.
- **Объектные типы** поддерживают наследование, инкапсуляцию и полиморфизм.

Идентификаторы объектов

- Каждый объект имеет *уникальный идентификатор объекта* — **OID** (object identifier), который создается и поддерживается системой автоматически.
- OID скрыт от пользователя и выполняет роль **первичного ключа**.
- В некоторых случаях объекты можно однозначно идентифицировать значениями заданного набора его свойств (атрибутов, связей или методов). Такие свойства объявляют **ключевыми**.
- Допускается явно указывать ключи, в том числе **составные** или **альтернативные**, при определении класса. Система поддерживает уникальность всех ключей класса.

Объекты и литералы

- Значения данных хранят объекты и литералы.
- **Объект** может изменять значение и характеризуется своим OID.
- **Литерал** – константа, которая может иметь сложную структуру, но не может изменять значение.
- Если значение литерала изменить, то получим другой литерал.

Связи классов

- Связи определяют между объектными типами.
- В объектной модели поддерживаются только **бинарные** связи, т. е. связи между двумя типами.
- Связи могут быть разновидностей
 - 1–1 (один-к-одному),
 - 1–М (один-ко-многим),
 - М–М (многие-ко-многим) в зависимости от того, сколько экземпляров соответствующего объектного типа может участвовать в связи.
- Для реализации N -арной связи ($N > 2$) создают отдельный класс, содержащий ссылки на объединяемые классы.

Экстент класса

- **Экстент** — множество объектов данного типа (класса) в базе данных.
- Если класс А является производным классом от базового класса В, экстент А — подмножество экстента В.
- Объект производного класса также является объектом базового класса.
- Если объект — аналог кортежа, экстент — аналог таблицы

Классы и интерфейсы

- **Класс** — объектный тип, на основе которого можно создавать объекты.
- Для класса допускается указывать экстенд и ключи.
- **Интерфейс** — абстрактный объектный тип, на основе которого нельзя создавать объекты. Его использует при формировании иерархии классов.
- Интерфейс не имеет экстенда и ключей.

Наследование

- Ациклический граф на основе *наследования*.
- Возможно множественное наследование.
- **Супертип** — базовый класс.
- **Подтип - производный** (подкласс).
- Подтип может переопределять свойства и операции супертипа, вводить новые свойства и операции.
- Механизм наследования от интерфейсов называют наследованием **IS–A** (как есть), а механизм наследования от классов — расширением (**extends**).
- При порождении подкласса от некоторого суперкласса подкласс наследует экстент и набор ключей суперкласса.

Атомарные и составные типы данных

- Объектная модель данных поддерживает две категории значений: объекты и литералы.
- **Литералы** являются константами (например, {"Hello", {1, 2, 3, 4}}).
- Среди литеральных типов данных можно выделить атомарные и составные типы.
- **Атомарные** — стандартные типы, например: *integer*, *float*, *string*, *boolean* и т. д.
- К **составным** относят структуры и коллекции.

Описание составных типов на языке ODL

- **Set** <T> — множество, где T — некоторый тип данных;
- **Bag** <T> — мультимножество, где T — некоторый тип данных;
- **List** <T> — упорядоченный список значений;
- **Array** <T, N> — массив с заранее определенным N количеством элементов, где T — некоторый тип данных;
- **Dictionary** <S, T> — словарь (ассоциативный массив), поименованный массив элементов, где S — тип ключа, T — тип значения;
- **Struct** {тип имя, тип2 имя2, ... } — структура, обращение идет по именам полей. У каждого поля свое название.

Описание классов на языке ODL

- Язык ODL используют для описания интерфейса базы данных. Реализация базы данных выполняется на языке OML.
- Описание объектного типа (класса) состоит из следующих элементов:
 - **имя** (название) класса;
 - набор **супертипов** (базовых классов или интерфейсов);
 - имя поддерживаемого системой **экстента** (extent);
 - один или более **ключей** (key или keys);
 - набор **атрибутов**, каждый из которых может быть объектом или литеральным значением;
 - набор **связей**, каждая из которых указывает на некоторый другой объект или множество объектов;
 - набор **методов**, определяемых своими сигнатурами.

Конструкция для описания класса на языке ODL

```
class название  
    (extent название_экстента key ключи)  
{  
    атрибуты;  
    методы;  
    связи;  
};
```

Для возможности создания объектов класса
обязательно указывают его экстент.

Объявление ключей класса на языке ODL

- Ключи указывать не обязательно, поскольку по умолчанию создается OID, который назначается всегда, даже если объявлен свой ключ.
- Составной ключ задают перечислением полей внутри скобок
key (поле1, поле2, ...)
- где поле — атрибут, метод или связь данного класса.
- Альтернативные ключи задают через запятую
key поле1, поле2, ...

Объявление интерфейса на языке ODL

- Определение класса отличается от определения интерфейса наличием необязательных разделов: **extends** (наследование), **extent** (экстент) и **key** (ключ(и)).
- В остальном описание класса и интерфейса совпадают:

```
interface название  
{  
    атрибуты  
    методы  
    СВЯЗИ  
};
```

Объявление наследования на языке ODL

- Наследование класса A от класса B задают с помощью ключевого слова

Class A extends B (...)

- Множественное наследование задают перечислением базовых классов через запятую

Class A extends B: C, D, E (...)

- Только первый из базовых типов может быть классом (B), остальные (C, D, E) — только интерфейсами.

Определение атрибутов

- Для задания атрибута указывают ключевое слово **attribute**, его **название** и **тип**, например:

Class Фильм (extent Фильмы key (Год, Название))

{

attribute string Название;

attribute integer Год;

attribute Bag<string> Награды;

attribute Struct{ string Режиссер,

List<string> Ассистенты} Создатели;

}

- Типом атрибута может быть любой составной или атомарный тип.

Определение связей

- Для задания связи используются ключевые слова:
 - **relationship** — ключевое слово для определения связи;
 - **inverse** — ключевое слово для отображения обратной связи.
- Связь обязательно прописывают в обоих классах и указывают два именованных конца.
- Перемещаться по ней можно в обоих направлениях (в отличие от ссылок в объектно-реляционной модели),
- Необходимо отследить соответствие описаний связей в обоих классах.

Пример описания связи 1-M

```
class Фильм (...)  
{  
    .....  
    relationship Студия made  
        inverse Студия::owns;  
}  
  
class Студия (...)  
{  
    ...  
    relationship Set<Фильм> owns  
        inverse Фильм::made  
}
```

Параметры описания связи

relationship тип название1

inverse тип::название2

- где тип — класс, с которым данный класс связан;
- **название1** — поле текущего класса, являющегося ссылкой на объект другого класса;
- **название2** — поле связанного класса, являющегося ссылкой на объект данного класса.
- Если ссылка указывает на один объект класса, то записывают название класса.
- Если ссылка указывает на множество объектов, то это коллекция ссылок и записывают конструктор коллекции, примененный к типу класса, например, **Set <тип>**.
- Допускается использовать любую коллекцию, один раз примененную к классу.

Описание методов

- При определении класса задают сигнатуры его методов (операций).
- Сигнатура содержит название метода, тип возвращаемого значения, перечень параметров и перечень исключений, например:

**string GetActorName(in integer RoleTime)
raises (noAct,manyAct)**

- где **raises** — перечисляет исключения, вызываемые методом.
- При описании параметров метода указывают способ их передачи:
 - In — по значению (входной), по умолчанию;
 - Inout — по ссылке;
 - Out — по ссылке (выходной).
- Тело метода описывают при реализации в среде программирования.

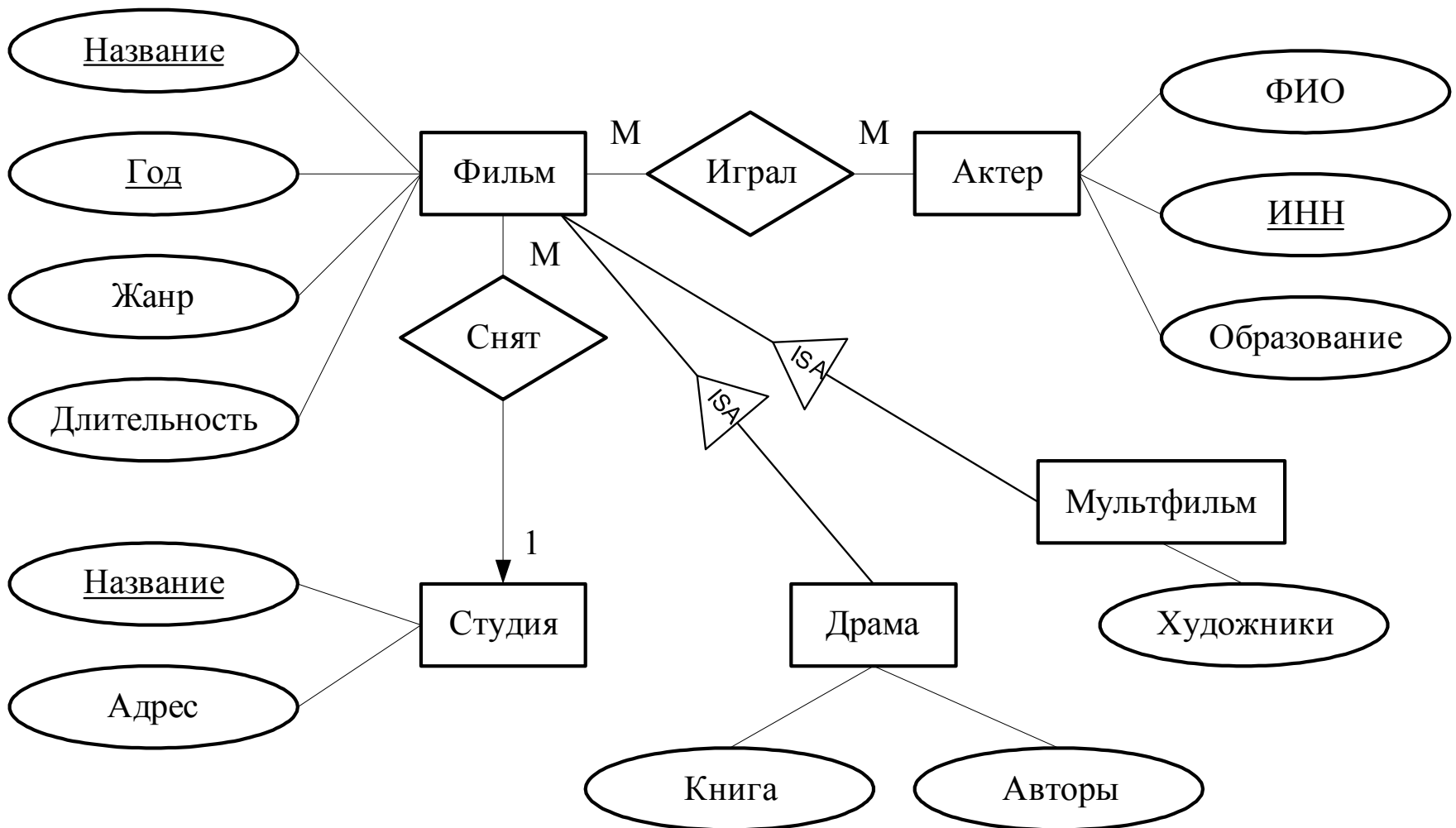
Правила преобразования ER модели в объектную модель

- При переходе к объектной модели БД элементы модели сущность—связь преобразуют по следующим правилам:
 - сущность — в класс;
 - атрибут сущности — в атрибут класса;
 - ключ сущности — в ключ класса;

Преобразование связей

- **бинарная** связь (1–М, 1–1 или М–М) — в связь классов, причем в каждый класс добавляется одна ссылка или коллекция ссылок на связанный класс;
- **N-арная** связь ($N > 2$) — в новый класс, который содержит по одной ссылке на каждый класс, участвующий в связи, а те классы будут содержать коллекции ссылок на новый класс;
- **слабая сущность и поддерживающие ее связи** — в класс, содержащее атрибуты слабой сущности и ссылки (связи) на все поддерживающих ее классы. Ее ключом будет совокупность собственного ключа и этих ссылок;
- **связь ISA** — в иерархию наследования классов, причем, базовая сущность станет суперклассом, а производная — подклассом.

Пример ER-модели



Описание класса Фильм

```
class Film(extent Films key(name, year))
{
    attribute string name;
    attribute integer year;
    attribute integer len;
    attribute enum Ftype {bw, color} type;

    relationship Studia stud
    inverse Studia::fs;
    relationship Set<Actor> acts
    inverse Actor::infs;

    integer ActCount() raises(noActors);
    void FILMSInYear(in integer year, out Set<Films>)
    raises(noFilms, badYear);
};
```

Описание класса Студия

```
class Studio (extent Studies)  
{  
  attribute string sname;  
  attribute Struct Addr  
  {  
    string city,  
    string street  
  } addr;  
  
  relationship Set<Film> fs  
    inverse Film::stud;  
};
```


Описание класса Актер

```
class Actor(extent Actors key inn)
{
    attribute string inn;
    attribute string fio;
    attribute List<string> edu;

    relationship Set<Film> infs
        inverse Film::acts;
};
```

Описание производных классов

```
class MF extends Film (extent MFS)
```

```
{  
    attribute string drawer;  
};
```

```
class Drama extends Film (extent Dramas)
```

```
{  
    attribute Struct {  
        Set<string> authors,  
        string bname } book;  
};
```

Запросы на языке OQL

- сходная с языком **SQL** нотацию для выполнения запросов к объектной БД из программы на объектно-ориентированном языке высокого уровня.
- задает **стандарт** для расширения ООЯП функциями по извлечению данных из БД и связыванию их с переменными программы.
- Конкретный язык программирования, соответствующий стандарту, должен обеспечить **заданные** стандартом возможности, но может иметь собственный синтаксис.
- Основным оператором языка OQL является команда **SELECT** для выборки данных.
- Команды **добавления, изменения и удаления** данных в язык OQL не входят и реализуются функциями объектно-ориентированного языка программирования (языком OML).

Точечная нотация

- Для доступа к компонентам переменных со сложным типом используют точечную нотацию.
- если p обозначает объект, a — некоторое свойство класса (атрибут, связь или метод), для доступа к свойству a объекта p пишут

$p.a$

- В точечной нотации для перехода к связанному объекту допускается указывать точку («.») или стрелку («→»). В языке OQL стрелка — синоним точки.
- При использовании точечной нотации допустим любой уровень вложенности .

Пример класса для иллюстрации точечной нотации

Class T (extent PD)

{

Attribute integer **a**;

Relationship classB **myb**

inverse classB :: myT;

string fun();

classV fun2();

};

Пример точечной нотации

- *если p — объект класса T*
- **$p.a$** — значение этого атрибута в объекте p ;
- **$p.myb$** — объект (или множество объектов), соединенных с p связью myb ;
- **$p.fun()$** или **$p.fun(\text{параметры})$** — результат выполнения метода fun ;
- **$p.myb.b1$** — атрибут $b1$ объекта класса $classB$, на который ссылается атрибут p по ссылке myb ;
- **$p.myb.b2()$** — вызов метода $b2()$ объекта класса $classB$, на который ссылается атрибут p по ссылке myb ;
- **$p.fun2().c1$** — атрибут $c1$ объекта класса $classV$, возвращаемого методом $fun2()$.

Запрос выборки с условием

- конструкции **SELECT— FROM— WHERE**,
- двойные кавычки для строк,
- для оператора «не равно» применяется символ **!=**
- отличие от SQL - секция **FROM**
FROM Films AS m
- ключевое слово **AS** применяется по выбору.
- выражение **Movies m** означает, что **m** — переменная, указывающая на каждый объект из экстента **Movies**.
- Пример выборки всех объектов фильмов из экстента **Films**

SELECT m FROM Films m

Переход по связи от М к 1

- названия фильмов и выпустивших их студий, расположенных в Москве

```
SELECT f.name,  
        f.stud.sname  
FROM Films f  
WHERE  
        f.stud.addr.city = “Москва”
```


Переход по связи от 1 к M

- названия фильмов и выпустивших их студий, расположенных в Москве (запрос можно рассматривать как выполнение вложенных циклов).

```
SELECT f.name,  
        f.stud.sname  
FROM Studies s,  
        s.fs  f  
WHERE  
        s.addr.city = “Москва”
```

Переход по связи М-М

- Ф.И.О актеров, игравших в фильме остров

```
SELECT    a.fio  
FROM      Actors a, a.infs r  
WHERE  
          r.name = “Остров”
```

Изменение типа результатов запроса

- Bag<string> (мультимножество)
SELECT m.name FROM Films m
- Set<string> (множество)
**SELECT DISTINCT m.name
FROM Films m**
- List<string> — если нужен упорядоченный список, то используется фраза ORDER BY:
**SELECT m.name FROM Films m
ORDER BY m.year**

Приведение типа результатов запроса

- Пример, порождающий множество или мультимножество структур (выбор ремейковых фильмов):

```
SELECT distinct Struct (A1: S1, A2:S2)  
FROM Films S1, Films S2  
WHERE S1.name = S2.name
```

- Задаем названия полей A1 и A2 и получаем в результате структуру:

```
Set <Struct { Film A1, Film A2} >
```

Логические множественные условия

FOR ALL x IN S : $C(x)$

- результат вычисления выражения равен TRUE, если каждый элемент x в коллекции S удовлетворяет условию $C(x)$, и FALSE в противном случае (квантор всеобщности);

EXISTS x IN S : $C(x)$

- результат вычисления выражения равен TRUE, если существует по меньшей мере один элемент, для которого условие $C(x)$ выполняется, и FALSE в противном случае (квантор существования).

Пример логического множественного условия

- Актеры, которые снимались только на студии Мосфильм:

```
SELECT a.fio  
FROM Actors a  
WHERE for all f in a.infs:  
    f.stud.sname="Мосфильм"
```

Операторы агрегирования

- В языке OQL используют те же операторы агрегации, что и в языке SQL.
- В языке SQL их применяют к столбцу таблицы, а в языке OQL — к коллекции с членами подходящего типа:
 - **COUNT** применяют к любой коллекции;
 - **SUM** и **AVG** — к коллекции арифметических типов,
 - **MIN** и **MAX** — к коллекции любых типов, которые можно сравнивать, например, к строкам.
- Например, для вычисления средней продолжительности всех фильмов создадим мультимножество длительности всех фильмов:
AVG(SELECT m.length FROM Films m)

Группировка

- Для группировки в языке OQL используют оператор **GROUP BY**, который содержит следующие элементы:
- ключевые слова GROUP BY;
- список разделенных запятыми атрибутов разбиения, каждый из которых включает в себя: группирующее выражение, двоеточие, имя выражения. Например,

GROUP BY title: m.name, fy:m.year

Промежуточный результат группировки

- Реальное значение, возвращаемое оператором GROUP BY представляет собой множество структур . Члены этого множества имеют форму:

Struct (title: m.name, fy:m.year, P : partition)

- Все поля кроме последнего поля обозначают группу и содержат значения группирующих выражений для этой группы.
- Последнее поле имеет специальное имя **partition** и содержит мультимножество объектов *m*, попавших в эту группу.

Выборка с группировкой

- Из секции SELECT запроса, содержащего оператор GROUP BY, можно обращаться к полям результата GROUP BY, а именно title, fu и partition.
- С помощью **partition** можно ссылаться на объекты t, являющиеся ее членами.
- Ссылаться на переменную t, входящую в секцию FROM, можно только внутри оператора агрегации, воздействующего на partition.

Пример группировки

- Приведем пример выборки Ф.И.О актеров и суммарной длительности фильмов, в которых они играли:

```
SELECT act, yr ,  
        sumlen: SUM (SELECT p. r. length  
                      FROM partition p)  
FROM Films r, r.acts a  
GROUP BY act: a.fio, yr: r.year
```

Группировка с условием

- В языке OQL за оператором GROUP BY может следовать оператор HAVING с таким же значением, как и HAVING в языке SQL.
- Оператор вида

HAVING <условие>

- служит для устранения некоторых групп, созданных оператором GROUP BY.
- Условие относится к значению **partition** каждой группы, полученной в результате выполнения оператора GROUP BY.
- При выполнении условия эта группа передается в результирующий набор; в противном случае она не используется в результате запроса.

Пример группировки с условием

- Пример ограничения списка актеров теми, кто снимался в фильмах длительностью >120

```
SELECT act, yr ,  
       sumlen: SUM (SELECT p. r. length  
                     FROM partition p)  
FROM Films r, r.acts a  
GROUP BY act: a.fio, yr: r.year  
HAVING MAX (SELECT p.r.length  
             FROM partition p) > 120
```

Операции над коллекциями

- К двум объектам типа множеств или мультимножеств можно применять операторы объединения, пересечения и разности, которые, как и в языке SQL, выражаются ключевыми словами **UNION**, **INTERSECT** и **EXCEPT** соответственно.

(select) UNION (select

Встраивание OQL в ООЯВУ

```
List<Film> lst =  
    select o from  
    Films o where o.year = 2021;  
  
foreach( f in lst)  
{  
    // обработка f  
    print f.name;  
}
```

CRUD в OML

```
Film f = new Film();
```

```
f.name = "Терминатор";
```

```
f.stud = select s from Studies where  
    sname="Мосфильм";
```

```
...
```

```
Film f = select o from Films o where ...;
```

```
f.Year = 2001;
```

```
....
```

```
Delete f;
```


Пример 1

- Фильмы, выпущенные до 1950 г., где снимался Иванов И.И.

```
SELECT f.name  
FROM Films f, f.acts a  
WHERE a.fio="Иванов И.И."  
      AND f.year < 1950
```

Пример 2

- Получить список по студиям (название, количество фильмов, количество актеров), выпустившим более 10 фильмов, с сортировкой по названию

```
SELECT name, cntFilm : COUNT(SELECT p.f  
                                FROM partition p),  
        cntAct : COUNT(SELECT p.a  
                        FROM partition p)  
FROM Studies s, s.fs f, f.acts a  
WHERE COUNT(s.fs)>10  
GROUP BY name:s.sname  
ORDER BY name
```

Спасибо за внимание!

Виноградова Мария Валерьевна

Vinogradova.m@bmstu.ru

МГТУ им. Н.Э. Баумана
кафедра Систем обработки информации и
управления (ИУ5)